QDROP: RANDOMLY DROPPING QUANTIZATION FOR EXTREMELY LOW-BIT POST-TRAINING QUANTIZATION

26/04/07

[기존 문제]

Algorithm 1: BRECQ optimization

Input: Pretrained FP model; Calibration dataset, iteration T
for *all* $i = 1, 2, \ldots, N$*-th block in the FP model* **do**
  Collect input data to the block $\mathbf{x}^{(i)}$, the FP output $\mathbf{z}^{(i)}$ and its gradient $\mathbf{g}^{(\mathbf{z}^{(i)})}$;
  for *all* $j = 1, 2, \ldots, T$*-iteration* **do**
    Get quantized output $\hat{\mathbf{z}}^{(i)}$ and compute $\Delta\mathbf{z}^{(i)} = \mathbf{z}^{(i)} - \hat{\mathbf{z}}^{(i)}$;
    Descend Eq. (10) and update the rounding of all the weights in this block (Eq. (16));
    if *Activation Quantization is triggered* **then**
      Update the activation quantization step size (Eq. (18)).
  After optimization, compute the sensitivity for each layer and between layers (2-bit only);
return Quantized model, Sensitivities for mixed precision;

BRECQ 알고리즘을 보면, weight quantization Eq.16 이후, activation quantization Eq.18이 각각 따로 quantization을 진행

$$\min_{\hat{\boldsymbol{w}}} \mathbb{E}\left[L(\hat{\boldsymbol{w}}, \boldsymbol{x}) - L(\boldsymbol{w}, \boldsymbol{x})\right] \approx \min_{\hat{\boldsymbol{w}}} \mathbb{E}\left[\frac{1}{2}\Delta\boldsymbol{w}^\top \mathbf{H}^{\boldsymbol{w}} \Delta\boldsymbol{w}\right], \tag{2}$$

where $\mathbf{H}^{\boldsymbol{w}} = \mathbb{E}_{\boldsymbol{x}\sim\mathcal{D}_t} \nabla^2_{\boldsymbol{w}} L(\boldsymbol{w}, \boldsymbol{x})$ is the expected second-order derivative. The above objective could be transformed into the change of output weighted by the output Hessian.

$$\min_{\hat{\boldsymbol{w}}} \mathbb{E}\left[\Delta\boldsymbol{w}^\top \mathbf{H}^{\boldsymbol{w}} \Delta\boldsymbol{w}\right] \approx \min_{\hat{\boldsymbol{w}}} \mathbb{E}\left[\Delta\boldsymbol{a}^\top \mathbf{H}^{\boldsymbol{a}} \Delta\boldsymbol{a}\right] \tag{3}$$

About the above minimization, they finetune only the weight by reconstructing each block/layer output (See Case 1 in Fig. 1). But they did not explore the activation quantization during output reconstruction with only modeling weight quantization as noise. The step size for activation quantization is determined after the reconstruction stage.

기존 PTQ에서는 이처럼, activation quantization에 따른 오차를 weight quantization시 고려하지 않음

Question)

Activation이 Full-Precision일 때와, 2,3-bit일 때의 Weight가 서로 달라야 되지 않을까?

즉, activation quantization에 의한 noise도 weight quantization에 반영해야 한다.

이에 본 논문은, **effect of activation quantization in PTQ**를 실험적으로, 이론적으로 증명하고 이를 반영하여 QDrop이라는 activation-aware PTQ를 제안

[Method]

[실험적 관측]

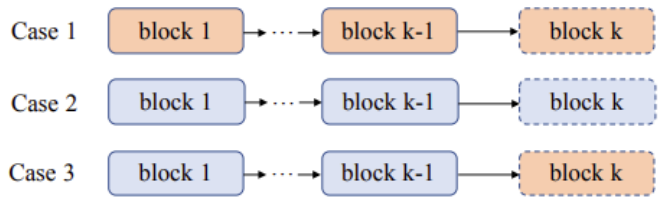


Figure 1: 3 cases to involve activation quantization when optimizing the k_{th} block's weight rounding. Activations are quantized inside the blue block and not quantized inside the orange block.

Case	1	2	3
ResNet-18	18.88	45.74	48.07
ResNet-50	4.34	46.98	49.07
MobileNetV2	5.83	50.71	51.20
RegNet-600MF	42.77	60.94	62.07
MnasNet	26.62	58.79	60.19

Table 1: 2-bit or 3-bit post-training quantization accuracy on ImageNet dataset across different cases and different models.

Case1: 기존 방식 (x activation quantization)

Case2: Full activation quantization

Case3: 현재 block 앞에만 activation quantization X

Case1 보다 Case2가 성능이 좋았다. 즉, activation quantization을 적용해야 한다.

Case2 보다 Case3의 성능이 좋았다. 무조건 적용하는 것이 아닌 **어떻게 적용해야** 하는 지에 대한 필요성 발견

이를 통해 실험적으로 activation quantization을 반영하는 것의 당위성을 관측했다.

그렇다면, 왜 더 좋은지, Weight Quantization에 어떠한 영향을 미치는 지에 대한 분석을 진행

[이론적 분석]

1) Activation error 수식 변환

To eliminate the influence induced by the range of activations, we take the noise of multiplicative form: $\hat{a} = (1+u) \cdot a$. In the quantization background, we denote the fixed-point integer value with respect to a as $\bar{a}$ ($a = (\bar{a}+c) \cdot s$ and $\hat{a} = \bar{a} \cdot s$). And then u can be denoted as:

$$\begin{aligned} u &= \frac{\hat{a}}{a} - 1 \\ &= \frac{\bar{a} \cdot s}{(\bar{a}+c) \cdot s} - 1 \\ &= \frac{\bar{a}}{\bar{a}+c} - 1 \\ &= \frac{-c}{\bar{a}+c}. \end{aligned} \tag{10}$$

activation error 를 scale과 range에 영향을 받지 않도록 곱으로 환산. **e = au**

2) Activation noise aware PTQ objective

Conventionally, the activation quantization could be modeled as injecting some form of noise imposed on the full-precision counterpart, defined as $e = (\hat{a} - a)$. To remove the influence of activation range on e, we translate the noise into a multiplicative form, i.e., $\hat{a} = a \cdot (1 + u)$, where the range of u is affected by bit-width and rounding error. Detailed illustration of the new form noise can be found in Appendix A.

Here, $\mathbf{1} + \boldsymbol{u}(\boldsymbol{x})$ is adopted to present the activation noise since it is related to specific input data point $\boldsymbol{x}$. Equipped with the noise, we add another argument in calculating the loss function and define our optimization objective in PTQ as:

$$\min_{\hat{\boldsymbol{w}}} \mathbb{E}_{\boldsymbol{x} \sim \mathcal{D}_c}[L(\boldsymbol{w} + \Delta \boldsymbol{w}, \boldsymbol{x}, \mathbf{1} + \boldsymbol{u}(\boldsymbol{x})) - L(\boldsymbol{w}, \boldsymbol{x}, \mathbf{1})]. \tag{4}$$

a^ = a + e = a + au = a(1+u)

이를 통해, 위와 같은 PTQ objective 재구성 (activation noise 항 추가)

3) Activation noise → Weight perturbation (가중치 섭동)

We hereby use a transformation that can absorb the noise on activation and transfer to weight, where the perturbation on weight is denoted as $\mathbf{1} + \boldsymbol{v}(\boldsymbol{x})$ ($\boldsymbol{V}(\boldsymbol{x})$ is used in matrix multiplication format). Consider a simple matrix-vector multiplication $\boldsymbol{W}\boldsymbol{a}$ in forward pass, we have $\boldsymbol{W}(\boldsymbol{a} \odot (\mathbf{1} + \boldsymbol{u}(\boldsymbol{x}))) = (\boldsymbol{W} \odot (\mathbf{1} + \boldsymbol{V}(\boldsymbol{x})))\boldsymbol{a}$, given by

$$\boldsymbol{W}(\boldsymbol{a} \odot \begin{bmatrix} 1 + \boldsymbol{u}_1(\boldsymbol{x}) \\ 1 + \boldsymbol{u}_2(\boldsymbol{x}) \\ \dots \\ 1 + \boldsymbol{u}_n(\boldsymbol{x}) \end{bmatrix}) = (\boldsymbol{W} \odot \boxed{\begin{bmatrix} 1 + \boldsymbol{u}_1(\boldsymbol{x}) & 1 + \boldsymbol{u}_2(\boldsymbol{x}) & \dots & 1 + \boldsymbol{u}_n(\boldsymbol{x}) \\ 1 + \boldsymbol{u}_1(\boldsymbol{x}) & 1 + \boldsymbol{u}_2(\boldsymbol{x}) & \dots & 1 + \boldsymbol{u}_n(\boldsymbol{x}) \\ \dots \\ 1 + \boldsymbol{u}_1(\boldsymbol{x}) & 1 + \boldsymbol{u}_2(\boldsymbol{x}) & \dots & 1 + \boldsymbol{u}_n(\boldsymbol{x}) \end{bmatrix}})\boldsymbol{a}. \tag{5}$$

By taking $\boldsymbol{V}_{i,j}(\boldsymbol{x}) = \boldsymbol{u}_j(\boldsymbol{x})$, quantization noise on the activation vector $(\mathbf{1} + \boldsymbol{u}(\boldsymbol{x}))$ can be transplanted into perturbation on weight $(\mathbf{1} + \boldsymbol{v}(\boldsymbol{x}))$. Note that for a specific input data point $\boldsymbol{x}$, there are two distinct $\boldsymbol{u}(\boldsymbol{x})$ and $\boldsymbol{v}(\boldsymbol{x})$. Proof is available at Sec. B.1.

Activation에 곱해지는 u를 Weight에 곱해지는 V로 변환

즉, activation의 섭동을 weight의 섭동으로 변환

(위 과정은 단순 수학적 trick으로 activation noise를 weight의 섭동을 치환하기 위함)

Lemma 1. *For a quantized (convolutional) neural network, the influence of activation quantization on the final loss objective in post-training quantization can be transformed into weight perturbation.*

$$\mathbb{E}_{\boldsymbol{x} \sim \mathcal{D}_c}[L(\hat{\boldsymbol{w}}, \boldsymbol{x}, \mathbf{1} + \boldsymbol{u}(\boldsymbol{x})) - L(\boldsymbol{w}, \boldsymbol{x}, \mathbf{1})] \approx \mathbb{E}_{\boldsymbol{x} \sim \mathcal{D}_c}[L(\hat{\boldsymbol{w}} \odot (\mathbf{1} + \boldsymbol{v}(\boldsymbol{x})), \boldsymbol{x}, \mathbf{1}) - L(\boldsymbol{w}, \boldsymbol{x}, \mathbf{1})] \tag{6}$$

즉, activation quantization을 고려하여 가중치를 최적화하는 것은, weight가 perturbation, noise에 강건한 모델을 개발하는 것과 같다.

4) L(w^, x, 1) Interpolation

By interpolating $L(\hat{\boldsymbol{w}}, \boldsymbol{x}, \mathbf{1})$ into Lemma 1, we can obtain the final theorem:

Theorem 1. *For a neural network $\mathcal{G}$ with quantized weight $\hat{\boldsymbol{w}}$ and activation perturbation $\mathbf{1}+\boldsymbol{u}(\boldsymbol{x})$, we have:*

$$\mathbb{E}_{\boldsymbol{x}\sim\mathcal{D}_c}[L(\hat{\boldsymbol{w}}, \boldsymbol{x}, \mathbf{1}+\boldsymbol{u}(\boldsymbol{x})) - L(\boldsymbol{w}, \boldsymbol{x}, \mathbf{1})] \approx \mathbb{E}_{\boldsymbol{x}\sim\mathcal{D}_c}[\underbrace{(L(\hat{\boldsymbol{w}}, \boldsymbol{x}, \mathbf{1}) - L(\boldsymbol{w}, \boldsymbol{x}, \mathbf{1}))}_{(7-1)} + \underbrace{(L(\hat{\boldsymbol{w}} \odot (\mathbf{1}+\boldsymbol{v}(\boldsymbol{x})), \boldsymbol{x}, \mathbf{1}) - L(\hat{\boldsymbol{w}}, \boldsymbol{x}, \mathbf{1}))}_{(7-2)}]. \quad (7)$$

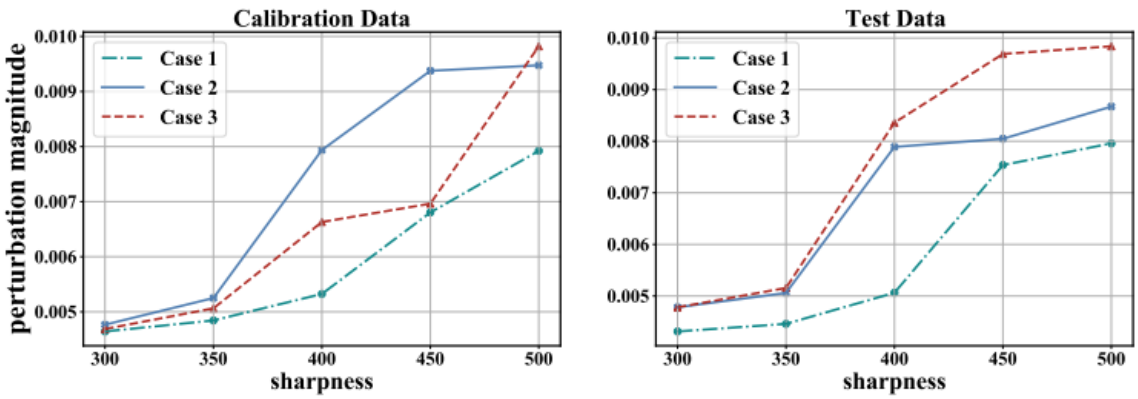
Here, Theorem 1 divides optimization objective into two terms. Term (7-1) is the same as Eq. (2) explored in (Nagel et al., 2020; Li et al., 2021a), which reveals how weight quantization interacts with loss function. Term (7-2) is the additional loss change by introducing activation quantization. In another way to interpret Eq. (7), the term (7-2) stands for the loss change with jitters on the weight quantized network $\mathcal{G}(\hat{\boldsymbol{w}}, \boldsymbol{x})$. This type of noise correlates with certain kinds of robustness.

7-1 항은 기존 가중치 양자화로 인한 Loss

7-2 항은 가중치에 추가적인 섭동 즉, activation quantization를 포함했을 때 (평탄도)

7-1: Calibration set에 맞게 quantization

7-2: Quantized wight에 섭동을 줄 때 강건하도록 (Flatness)



즉, Flat Local Minimum을 학습하자!

- ⇨ **Flat: 작은 noise에도 loss 변화가 적도록 (Robust: Generalization)**
- ⇨ **Case2는 sharp(Calibration data에 overfitting)해서, test에서는 성능 degrade**
- ⇨ **Flat local minimum은 calib에서는 비교적 떨어지더라도 general하게 test에서도 일정한 성능을 띄도록**

따라서, case3와 같이 매번 반영하지는 않도록 하여 perturbation에 강건하도록 하자

어떻게? QDrop

[QDrop]

Activation quantization을 반영하는 것이 더 flat한 분포를 학습한다.

하지만, case2와 case3를 참고할 때 매번하는 것보다 어떻게 반영하는지가 중요하다.

Case2 처럼 매번하면, activation noise가 calibration set에 overfitting.

QDrop은 아래와 같이 확률 p에 따라 반영하거나 반영하지 않는다.

QDROP. Inspired by this, we propose QDROP to further increase the flatness on as many directions as possible. In particular, we randomly disable and enable the quantization of the activation each forward pass:

$$\text{QDROP}: u = \begin{cases} 0 & \text{with probability } p \\ \frac{\hat{a}}{a} - 1 & \text{with probability } 1 - p \end{cases}. \quad (9)$$

반영 X
O

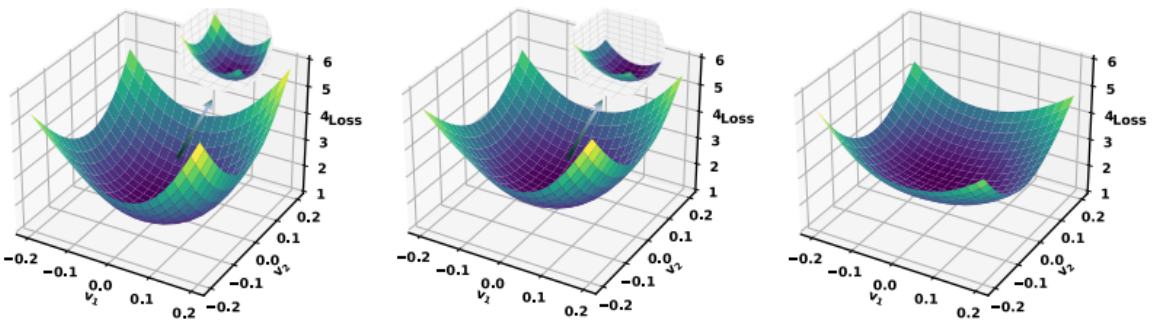
즉, activation error를 반영하여 weight quantization을 진행하면, calibration set에 weight quantization error를 줄이는데 overfitting 감소. (Weight perturbation)

하지만, 매번 진행할 경우, activation error를 줄이는데 calibration set에 overfit.(Case2)

이를 QDrop을 통해 dropout 처럼 overfitting에 대해 강건하도록 수정

Case1 < Case2 < Case3 < QDrop 순으로 (오른쪽이 good) 더 flat 함.

Discussions. QDROP can be viewed as a generalized form of the existing schemes. Case 1 and 2 respectively corresponds to the dropping probability of $p = 1$ and $p = 0$. Case 3 is equivalent to setting the block being optimized with dropping probability $p = 1$ and remains the quantization of



(a) Case 1 (b) Case 3 (c) QDROP

[QDrop Algorithm]

Algorithm 1: QDROP in one block for one batch

Input: the k_{th} block from layer i to layer j, a minibatch of quantized block input $\hat{\boldsymbol{a}}^{i-1}$, FP32 block input $\boldsymbol{a}^{i-1}$, FP32 block output $\boldsymbol{a}^{j}$, quantization dropping probability p.

{1. Forward propagation:}

During training phase, substitute $\hat{\boldsymbol{a}}^{i-1}$ with corresponding $\boldsymbol{a}^{i-1}$ at neuron-level with probability p and mark the replaced input as $\tilde{\boldsymbol{a}}^{i-1}$;

for $l = i$ *to* j **do**

  $\boldsymbol{a}^{l} \leftarrow \boldsymbol{W}^{l}\tilde{\boldsymbol{a}}^{l-1}$;

  $\hat{\boldsymbol{a}}^{l} \leftarrow \text{Quantize}(\boldsymbol{a}^{l})$;

  During training phase, randomly drop some $\hat{\boldsymbol{a}}^{l}$ with $\boldsymbol{a}^{l}$ as defined in Eq. (9) and get $\tilde{\boldsymbol{a}}^{l}$;

{2. Backward propagation:}

Compute $\Delta\boldsymbol{a}^{j} = \tilde{\boldsymbol{a}}^{j} - \boldsymbol{a}^{j}$;

Tune the weight by gradient descent ;

return Quantized block ;

a~ = a (원래 activation) 또는 a^ (Quantized activation)

[Experiment]

Method	Bits (W/A)	ResNet-18	ResNet-50	MobileNetV2	RegNet-600MF	RegNet-3.2GF	MNasNet-2.0
No Drop	2/2	46.64	47.90	4.55	25.52	39.76	9.51
QDROP	2/2	**51.14**	**54.74**	**8.46**	**38.90**	**52.36**	**22.70**
No Drop	2/4	64.16	69.60	51.61	61.52	70.29	60.00
QDROP	2/4	**64.66**	**70.08**	**52.92**	**63.10**	**70.95**	**62.36**

Table 2: Effect of QDROP.

Case2 처럼 모두하는 것보다 QDrop처럼 activation을 random하게 반영하는게 성능 좋다.

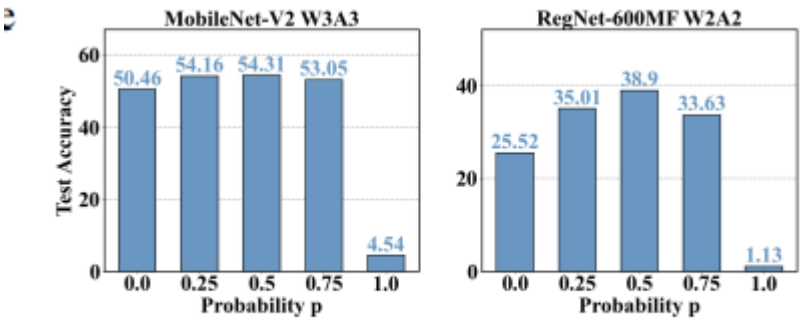


Figure 5: Impact of dropping probability on ImageNet.

p=0.5일 때 제일 성능이 좋음

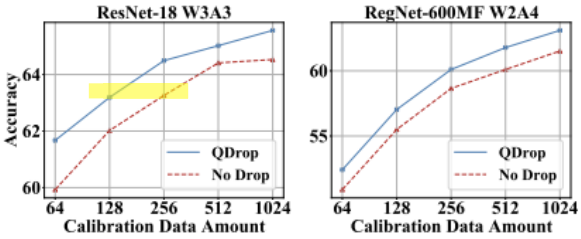


Figure 6: Impact of calibration data size on ImageNet.

Calibration	Bits (W/A)	No Drop	QDROP
MS COCO	4/4	68.64	**68.94**
MS COCO	3/3	64.12	**65.15**
CIFAR100	4/4	46.83	**52.88**
CIFAR100	3/3	21.74	**28.58**

Table 6: Cross domain data.

QDrop은 Robust하다.

적은 calibration set만으로도 더 좋은 성능을 띈다.

또한, cross domain calibration set으로도 더 좋은 성능을 띈다.

⇨ QDrop의 generalization (perturbation 효과)

[Conclusion]

QDrop은 activation error를 반영하여 weight quantization을 진행함으로써, **더 flat한 local minima**를 찾고, 더 나아가 randomly drop을 통해 calibration set에 overfitting되지 않고 이를 도달한다.